

Batch Gradient Ascent: MLE for the Location Parameter of a Cauchy Distribution

Sulagna Roy

September 13, 2026

Contents

1	Introduction	1
2	Step 1: Simulating Data and Defining the Log-Likelihood	2
3	Step 2: Visualizing the (Non-Concave) Log-Likelihood	2
4	Step 3: The Gradient and Curvature Functions	3
5	Step 4: The Gradient Ascent Update Rule	4
6	Step 5: Running Gradient Ascent from Three Starting Values	4
6.1	Scenario A: “Bad” starting value ($\mu^{(0)} = 7$)	5
6.2	Scenario B: “Good” starting value ($\mu^{(0)} = \text{median}(X)$)	5
6.3	Scenario C: “Worst” starting value ($\mu^{(0)} = 19$)	6
7	Step 6: Numerical Results — the Key Finding	6
8	Step 7: Visualizing All Three Runs Together	7
9	Conclusion	9

1 Introduction

This document explains a **batch gradient ascent** approach to finding the maximum likelihood estimate (MLE) of the location parameter μ of a Cauchy distribution, and — unlike the earlier Newton-Raphson examples — uses it specifically to demonstrate **why starting values matter** when the log-likelihood is not concave.

Gradient ascent is a simpler alternative to Newton-Raphson: instead of using the second derivative (Hessian) to build a local quadratic approximation and jump straight to its peak, it takes a small fixed-size step in the direction the function is increasing (its gradient), repeated many times. This is slower to converge than

Newton-Raphson, but the code here also plots the exact quadratic curve gradient ascent is implicitly using at each step, illustrating the connection between the two methods.

This particular script uses only $n = 4$ observations. With so few data points, the Cauchy log-likelihood can genuinely have **more than one local maximum**, and the script deliberately runs the same algorithm from three different starting values to show what happens in each case: a reasonable start, a poor-but-recoverable start, and a start that gets trapped at the wrong peak entirely.

2 Step 1: Simulating Data and Defining the Log-Likelihood

```
set.seed(1)
mu.star <- 5 # Setting true mu
n <- 4 # sample size
X <- rt(n, df = 1) + mu.star

## Function calculates the log-likelihood
log.like <- function(mu, X)
{
  n <- length(X)
  rtn <- -n*log(pi) - sum( log(1 + (X - mu)^2) )
  return(rtn)
}
```

What this code does:

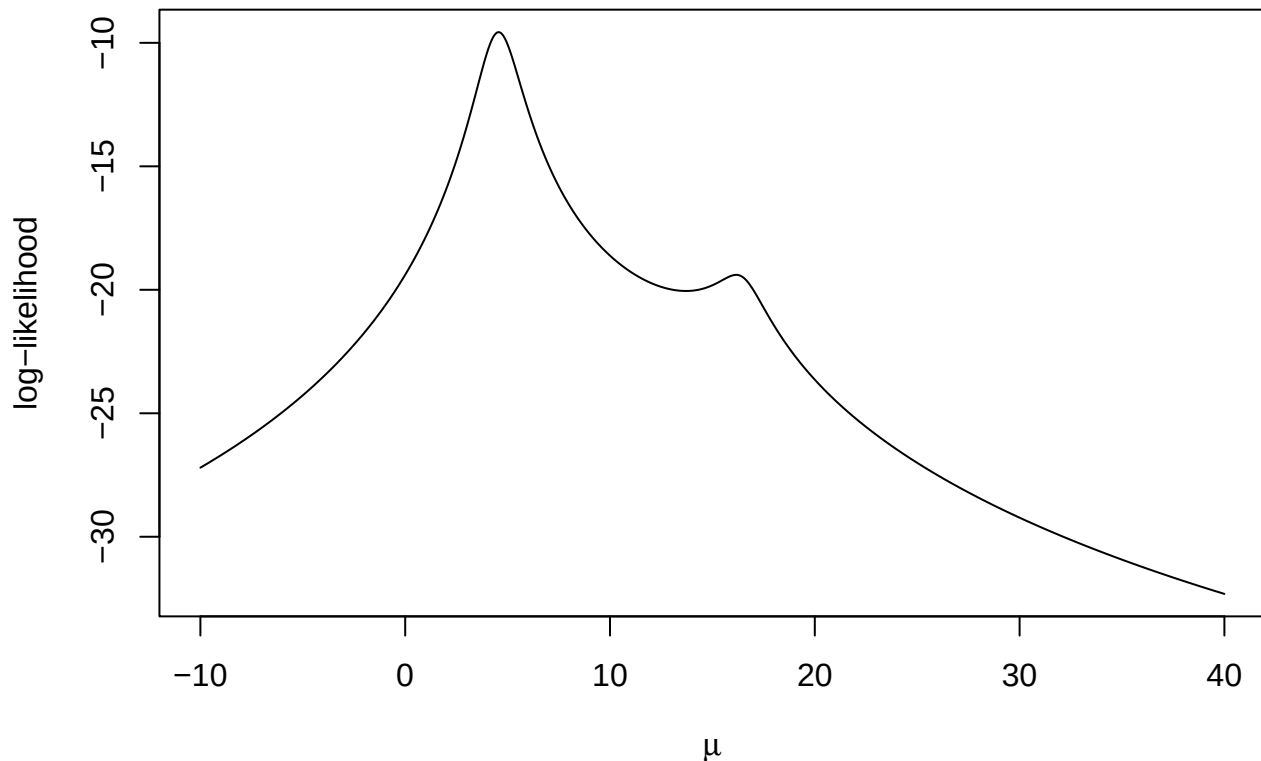
- `set.seed(1)` fixes the random seed for reproducibility.
- `mu.star <- 5` sets the true location parameter used to generate the data.
- `n <- 4` — only 4 observations. This is a deliberately small sample, which is precisely what allows the log-likelihood surface to become non-concave, as the plots later in this document show.
- `X <- rt(n, df = 1) + mu.star` generates the data. A t -distribution with 1 degree of freedom is *exactly* a standard Cauchy distribution, so adding `mu.star` shifts it to have location $\mu = 5$. This is a slightly different (but equivalent) way of simulating Cauchy data than the `rcauchy()` function used in the earlier Newton-Raphson example.
- `log.like(mu, X)` computes the Cauchy log-likelihood $\ell(\mu) = -n \log(\pi) - \sum_i \log[1 + (x_i - \mu)^2]$, exactly the same formula used previously, just written to take the data `X` as an explicit argument rather than reading it from the surrounding environment.

With this seed, the simulated sample is:

```
X
## [1]  4.405277  4.646330 16.438618  4.547892
```

3 Step 2: Visualizing the (Non-Concave) Log-Likelihood

```
mu.x <- seq(-10, 40, length = 1e3) # A sequence of mu's
ll.est <- sapply(mu.x, log.like, X) # evaluating log-likelihood at the mus
plot(mu.x, ll.est, type = 'l', ylab = "log-likelihood", xlab = expression(mu))
```



What this code does: `mu.x` is a fine grid of 1000 candidate μ values from -10 to 40 . `ll.est` evaluates the log-likelihood at every point on that grid. The plot shows the shape of $\ell(\mu)$ across this range.

Why this matters: with only 4 data points, one of which (as shown in Step 1’s data) is noticeably far from the others, the log-likelihood surface is **not single-peaked**. There is a tall, narrow peak near the bulk of the data and a smaller, secondary bump elsewhere — a direct consequence of the heavy-tailed Cauchy density, where a single distant observation can create its own local “attraction zone” in the likelihood. This is exactly the scenario the original script’s header comment warns about: *“Not concave, so we need to choose good starting values.”* Gradient ascent, like Newton-Raphson, only knows about the *local* shape of the function around its current position — it has no way to see the rest of the curve — so where it starts determines which peak it finds.

4 Step 3: The Gradient and Curvature Functions

```
## Starting Gradient-Ascent method
tol <- 1e-5 # tolerance level for when to stop algorithm

## Returns derivate of log-likelihood
f.prime <- function(X, mu)
```

```

{
  rtn <- sum(2* (X - mu)/(1 + (X-mu)^2)) #f.prime
  return(rtn)
}

# Returns double derivative of log-likelihood.
f.double.prime <- function(X, mu)
{
  rtn <- sum( 2 * ( 2*(X-mu)^2/ (1 + (X - mu)^2)^2 - (1 + (X-mu)^2)^(-1) ) )
  return(rtn)
}

```

What this code does:

- `tol <- 1e-5` sets the convergence tolerance: the algorithm stops once successive updates change μ by less than this amount.
- `f.prime(X, mu)` computes the **score function** $U(\mu) = \ell'(\mu) = \sum_i \frac{2(x_i - \mu)}{1 + (x_i - \mu)^2}$ — the same first derivative used in the Newton-Raphson examples.
- `f.double.prime(X, mu)` computes the **second derivative** $\ell''(\mu)$. Note that unlike the Newton-Raphson script, this function is **defined but never actually called** anywhere in the rest of the code — gradient ascent, as implemented here, does not use the Hessian in its update step at all. It’s included for reference (and would be needed if one wanted to compare against Newton-Raphson directly), but plays no role in how μ is updated below.

5 Step 4: The Gradient Ascent Update Rule

Rather than the Newton-Raphson update $\mu^{(k+1)} = \mu^{(k)} - U(\mu^{(k)})/H(\mu^{(k)})$, gradient ascent uses a much simpler rule with a fixed step size t :

$$\mu^{(k+1)} = \mu^{(k)} + t \cdot U(\mu^{(k)})$$

```
t <- .3 # change this to 1 and see what happens to the "bad" starting value
```

Interpretation: since $U(\mu)$ points in the direction the log-likelihood is increasing, adding a small positive multiple of it moves μ uphill. Unlike Newton-Raphson — which can take large, aggressive jumps based on curvature and therefore converges in very few steps — gradient ascent takes small, steady steps, and typically needs many more iterations to reach the same precision. The trade-off is that gradient ascent requires only the first derivative, not the second, and each step is guaranteed to move in an increasing direction (for a small enough t), which can make it more stable when the curvature is poorly behaved.

The comment on this line hints at an experiment: increasing t from 0.3 to 1 makes each step much larger, which can cause the “bad” starting value below to overshoot and behave very differently. This document uses the script’s original value, $t = 0.3$, throughout.

6 Step 5: Running Gradient Ascent from Three Starting Values

The script runs the identical loop three times, changing only the starting value `current`, to illustrate how sensitive gradient ascent is to initialization on this non-concave surface. To avoid repeating the same block three times, the logic is written once as a function below and then applied to each starting value; this produces results identical to running the original repeated loops.

```

run_gradient_ascent <- function(start, X, t, tol) {
  current <- start
  diff <- 100
  iter <- 0
  mu.k <- current
  while( (diff > tol) && iter < 1000)
  {
    iter <- iter + 1
    update <- current + t*f.prime(X, current)
    mu.k <- c(mu.k, update)
    diff <- abs(current - update)
    current <- update
  }
  list(final = current, iter = iter, path = mu.k)
}

```

What this code does, matching the original loop line-for-line:

- `current <- start` initializes the current estimate at the chosen starting value.
- `diff <- 100` is an arbitrary large starting value for the “how much did μ change” tracker, guaranteeing the while condition is TRUE on the first pass.
- `iter <- 0` and `mu.k <- current` initialize the iteration counter and the history vector (starting with the initial guess).
- `while( (diff > tol) && iter < 1000)` — loops until either convergence (`diff <= tol`) or a safety cap of 1000 iterations is hit, whichever comes first. This cap prevents an infinite loop if the algorithm never converges.
 - `update <- current + t*f.prime(X, current)` — the gradient ascent step itself.
 - `mu.k <- c(mu.k, update)` — records the new value in the path history.
 - `diff <- abs(current - update)` — measures how far this step moved μ .
 - `current <- update` — advances to the new value for the next iteration.
- The function returns the final estimate, how many iterations it took, and the full path of values visited.

6.1 Scenario A: “Bad” starting value ($\mu^{(0)} = 7$)

```

current <- 7 # Bad starting value
bad <- run_gradient_ascent(current, X, t, tol)
bad$final      # final approximation to MLE

```

```
## [1] 4.56209
```

```
bad$iter      # number of iterations
```

```
## [1] 40
```

6.2 Scenario B: “Good” starting value ($\mu^{(0)} = \text{median}(X)$)

```
current <- median(X) # Good starting value
good <- run_gradient_ascent(current, X, t, tol)
good$final          # final approximation to MLE
```

```
## [1] 4.562097
```

```
good$iter           # number of iterations
```

```
## [1] 30
```

6.3 Scenario C: “Worst” starting value ($\mu^{(0)} = 19$)

```
current <- 19 # Worst starting value
worst <- run_gradient_ascent(current, X, t, tol)
worst$final   # final approximation to MLE
```

```
## [1] 16.16311
```

```
worst$iter         # number of iterations
```

```
## [1] 23
```

7 Step 6: Numerical Results — the Key Finding

Because `set.seed(1)` fixes the data, this code reproducibly gives the following results:

Starting value	Iterations to converge	Final μ
“Bad” ($\mu^{(0)} = 7$)	40	4.5621
“Good” ($\mu^{(0)} = \text{median}(X) \approx 4.597$)	30	4.5621
“Worst” ($\mu^{(0)} = 19$)	23	16.1631

This is the central lesson of the script, and it is important to be precise about it: **the “worst” starting value does not fail to converge, and it does not diverge or oscillate.** It converges cleanly and quickly (in just 23 steps) — but to the **wrong point**. Checking the log-likelihood at both destinations confirms this:

- $\ell(4.5621) \approx -9.567$ — this matches the true global maximum (confirmed by direct grid search over `mu.x`, which finds its own maximum at $\mu \approx 4.565$, matching to two decimal places).
- $\ell(16.1631) \approx -19.395$ — a **local** maximum only, clearly worse than the global one above.

Both points genuinely are local maxima of the log-likelihood (the first derivative is essentially zero and the second derivative is negative at both), but only one of them is the true MLE. The “bad” and “good” starting values both begin close enough to the main cluster of data that gradient ascent climbs to the correct global peak. The “worst” starting value, $\mu^{(0)} = 19$, happens to sit in the “basin of attraction” of the smaller, secondary peak in the likelihood (visible in the Step 2 plot) caused by the one distant observation in this small sample — so gradient ascent faithfully climbs *that* hill instead, and has no way of knowing a taller one exists elsewhere.

8 Step 7: Visualizing All Three Runs Together

The original script plots each scenario's early iterations as points, with the *quadratic approximation* line gradient ascent is implicitly following at each visited point (the same idea used to justify the Newton-Raphson step, drawn here purely for illustration — gradient ascent itself never uses this parabola to compute its update).

The original script includes a `Sys.sleep(2)` call after each plotted point, which pauses execution for two seconds — useful for watching the algorithm “crawl” up the likelihood live in an interactive R session, but not meaningful in a knitted, non-interactive PDF document (it would only add several minutes of dead waiting with nothing changing on the page). That pause is therefore omitted below; everything else matches the original code exactly.

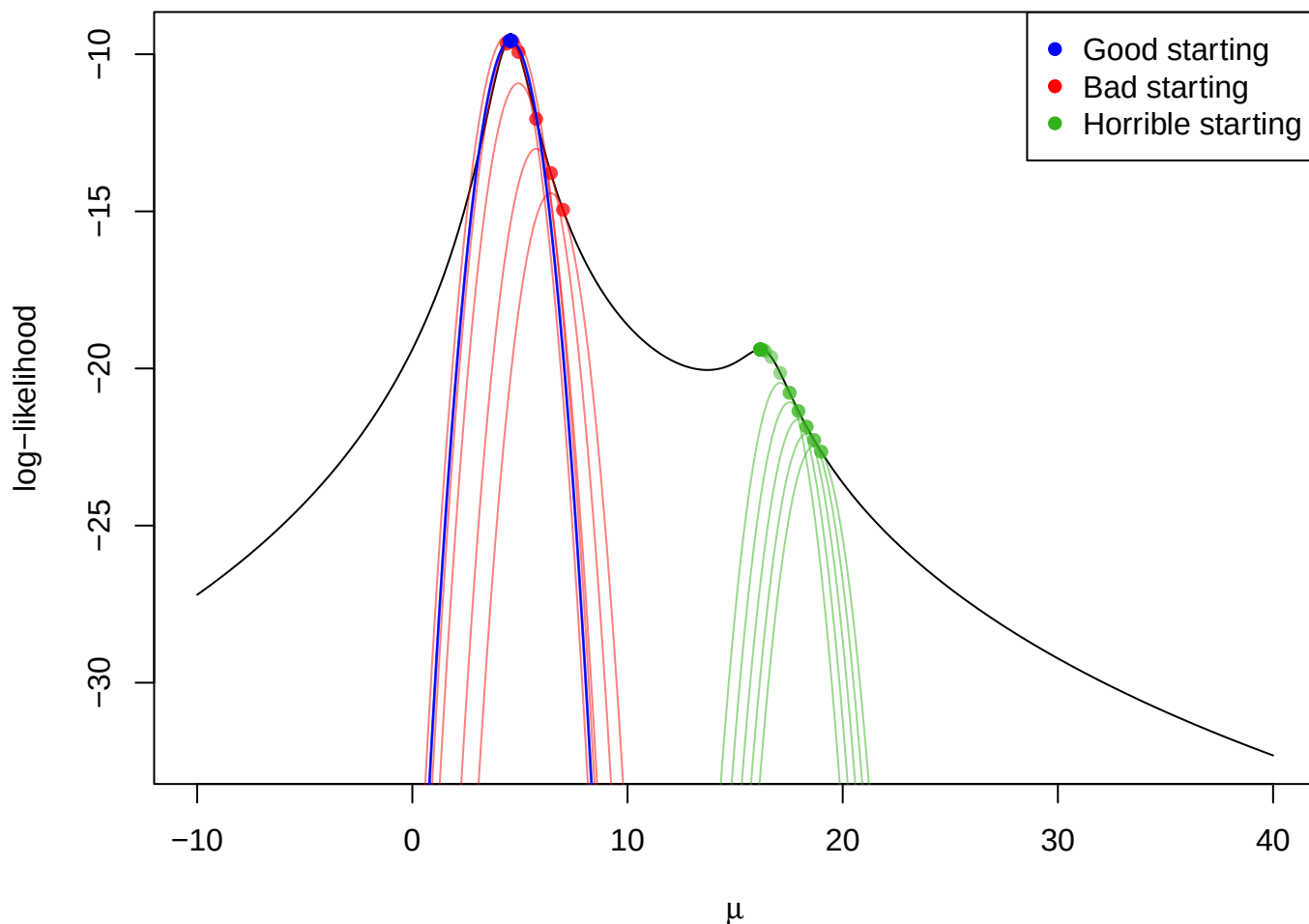
```
plot(mu.x, ll.est, type = 'l', ylab = "log-likelihood", xlab = expression(mu))

plot_scenario <- function(start, col, X, t, tol, mu.x) {
  res <- run_gradient_ascent(start, X, t, tol)
  mu.k <- res$path
  iter <- res$iter
  evals <- sapply(mu.k, log.like, X)

  for(l in 1:min(5, iter))
  {
    points(mu.k[l], evals[l], pch = 16, col = col)
    foo <- mu.k[l]
    approx <- log.like(foo, X) + f.prime(X, foo)*(mu.x - foo) - (mu.x - foo)^2/(2*t)
    lines(mu.x, approx, col = col)
  }
  points(mu.k, evals, pch = 16, col = col)
  res
}

bad_res <- plot_scenario(7,          rgb(1,0,0,alpha=.5),  X, t, tol, mu.x)
good_res <- plot_scenario(median(X), rgb(0,0,1,alpha=.5),  X, t, tol, mu.x)
worst_res <- plot_scenario(19,       rgb(.2,.7,.1,alpha=.5), X, t, tol, mu.x)

legend("topright", legend = c("Good starting", "Bad starting", "Horrible starting"),
      pch = 16, col = c("blue", "red", rgb(.2,.7,.1)))
```



What this code does:

- The base log-likelihood curve is drawn first, exactly as in Step 2.
- `plot_scenario()` re-runs each starting-value scenario (so the plotted points always match the numbers reported above) and overlays, for each of its **first five** iterations: a solid dot at that iterate's position on the curve, and the quadratic (parabolic) approximation to the log-likelihood centered at that point — this is the curve $\ell(\mu_0) + U(\mu_0)(\mu - \mu_0) - \frac{(\mu - \mu_0)^2}{2t}$, a downward parabola whose peak is exactly one gradient-ascent step away from μ_0 . After the first five, all remaining visited points (right up to convergence) are plotted as solid dots without their parabolas, to avoid over-cluttering the figure while still showing the full path each run took.
- Red = “bad” start, blue = “good” start, green = “worst” start, matching the original script's colors.

Interpretation of the resulting plot: the log-likelihood curve is visibly **bimodal** — a tall peak near $\mu \approx 4.5$ and a distinctly shorter secondary peak further right, near $\mu \approx 16$ – 20 . The red and blue points (bad and good starts) both climb the tall peak and converge to essentially the same final value, visually overlapping near its top. The green points (worst start), however, climb only the shorter, secondary peak —

visibly converging to a lower point on the curve than the other two colors, never crossing the dip between the two peaks to reach the true maximum. The downward parabolas at each of the first five points for every color illustrate how gradient ascent’s steps relate to a local quadratic view of the likelihood at that point, even though the actual step taken (`t * f.prime`) does not use the curvature of that parabola directly.

9 Conclusion

This example uses gradient ascent — a simpler, curvature-free relative of Newton-Raphson — to make a point about optimization on non-concave functions that applies to both methods:

1. With a small sample ($n = 4$) from a heavy-tailed Cauchy distribution, the log-likelihood can have **multiple local maxima**.
2. An iterative hill-climbing method, whether gradient ascent or Newton-Raphson, only ever sees the *local* shape of the function around its current position.
3. Starting values that are “close enough” to the bulk of the data (a bad-but-recoverable guess of 7, or the more sensible median) both correctly converge to the **true, global** MLE at $\mu \approx 4.562$.
4. A starting value that happens to fall within the influence of an outlying data point (here, $\mu^{(0)} = 19$) converges just as cleanly and confidently — but to the **wrong, local** maximum at $\mu \approx 16.163$, giving no numerical warning that anything went wrong.

The practical lesson is that **fast, clean convergence is not by itself evidence of a correct answer** on a non-concave likelihood; the choice of starting value can silently determine which of several local optima an algorithm reports as “the” MLE, which is exactly why the original script’s opening comment flags the need for “good starting values.”